

Docker and Kubernetes

Docker gets a container running on one machine. Kubernetes keeps many of them running across many machines. This part is the join between the two: what happens to our image once it leaves the registry.

1. Setting it up

Two things are needed: `kubectl`, and a cluster to point it at.

Cluster	Good for
Docker Desktop	One checkbox, already installed. The quickest start
<code>kind</code>	Runs the cluster in containers. Fast, and what CI usually uses
<code>minikube</code>	A local VM. More like a real node

```
kind create cluster --name dev
kubectl cluster-info
kubectl get nodes
```

```
kubectl config get-contexts          # which clusters kubectl knows
kubectl config use-context kind-dev
```

Check the context before every destructive command. `kubectl delete` does not ask, and the only thing between a local cluster and production is which context is selected.

The first deployment needs no YAML at all:

```
kubectl create deployment payroll --image=ghcr.io/acme/payroll:1.4
kubectl expose deployment payroll --port=8080
kubectl get pods
```

A locally built image is not visible to the cluster. The node pulls from a registry, and an image sitting in the laptop's Docker is not in one. On `kind` the fix is one command, and forgetting it is the most common first hour:

```
kind load docker-image payroll:1.4 --name dev
```

2. Docker against Kubernetes

They are not alternatives. One builds and runs a container; the other decides where containers run and keeps them running.

Docker	Kubernetes
Builds images	Does not build anything
Runs containers on one machine	Schedules containers across many
Restart is a policy on one container	Restart, reschedule and replace
Compose describes a stack for a host	Manifests describe desired state for a cluster
Scaling is <code>--scale</code> on one machine	Scaling adds pods on any node with room
Networking is a bridge on the host	A cluster network with DNS and load balancing

The one-sentence answer: Docker produces the image, Kubernetes runs it. They meet at the registry, and the image format is the contract between them.

3. Container runtime

Kubernetes does not run containers itself. The kubelet on each node asks a container runtime through an interface called the CRI.

```
kubelet → CRI → containerd → runc → the container
```

Runtime	Note
containerd	The usual default, and the same one Docker uses underneath
CRI-O	Built only for Kubernetes. Common on OpenShift
Docker Engine	Removed as a direct option in Kubernetes 1.24

"Kubernetes deprecated Docker" is the most misread headline in the field. What was removed was `dockershim`, the adapter that let the kubelet talk to Docker Engine. Kubernetes now talks to `containerd` directly, which Docker was using anyway.

Images built with Docker still work everywhere. The image format is the OCI specification, not a Docker format, so nothing about the Dockerfile, the build or the registry changed.

4. Kubernetes Pods

A pod is the smallest thing Kubernetes runs. It holds one container, usually, and can hold several that need to sit together.

```
apiVersion: v1
kind: Pod
metadata:
  name: payroll
spec:
  containers:
    - name: payroll
      image: ghcr.io/acme/payroll:1.4
      ports:
        - containerPort: 8080
```

Containers in one pod share a network namespace and can share volumes, so they reach each other on `localhost` and cannot both use port 8080.

Pattern	What it is for
Sidecar	A second container beside the application, such as a log shipper
Init container	Runs to completion first, for a migration or a wait

A bare pod is not what gets deployed. Nothing recreates it if the node dies. Pods are created by a Deployment, which is what section 6 is about.

Pods are disposable and their addresses change. Anything written to a pod's filesystem is gone when it is replaced, and nothing should ever hold a pod's IP address.

5. Images in Kubernetes

The `image` field takes exactly the reference `docker pull` would take.

```
image: ghcr.io/acme/payroll:1.4
image: ghcr.io/acme/payroll@sha256:4a1c8f3b9e2d7c6a5b4f3e2d1c0b9a8f
```

The node does the pulling, not us. That has two results. A private registry needs credentials on the cluster, and an image only on a laptop cannot be run.

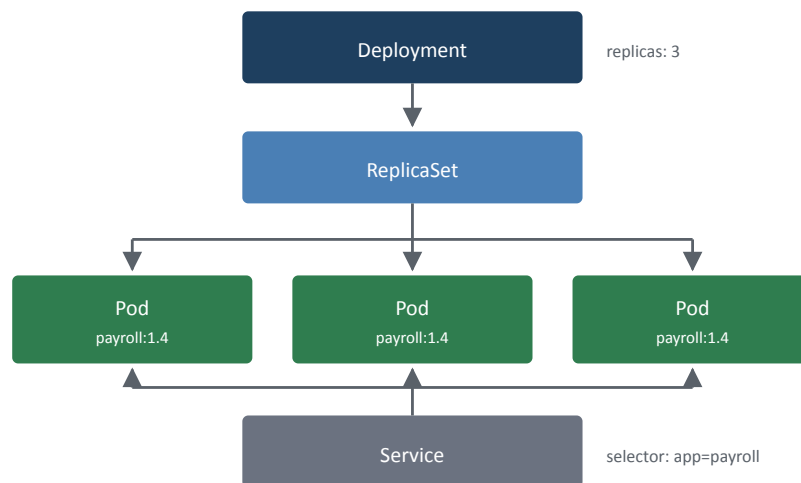
```
kubectl create secret docker-registry ghcr \
  --docker-server=ghcr.io \
  --docker-username=acme-ci \
  --docker-password=$TOKEN
```

```
spec:
  imagePullSecrets:
    - name: ghcr
  containers:
    - name: payroll
      image: ghcr.io/acme/payroll@sha256:4a1c8f3b9e2d...
```

Deploy the digest. A tag is resolved by each node separately, at whatever moment that node pulls, so a moving tag can leave two pods of one Deployment running different code. A digest cannot.

6. Deployments

A Deployment states what should be running. Kubernetes makes it so, and keeps it so. It does that through two objects rather than one.



The Deployment owns a ReplicaSet, which owns the pods. The Service finds the same pods by label, not by address.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payroll
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payroll
```

The rest of it is the pod template, which is the section 4 pod with its labels attached:

```
template:
  metadata:
    labels:
      app: payroll
  spec:
    containers:
      - name: payroll
        image: ghcr.io/acme/payroll:1.4
        ports:
          - containerPort: 8080
```

selector and the template labels must match. They are what ties the Deployment to its pods, and a mismatch is rejected rather than guessed at.

Changing the image starts a rolling update: new pods come up, old ones go away, and the Service keeps serving throughout.

```
kubectl set image deployment/payroll payroll=ghcr.io/acme/payroll:1.5
kubectl rollout status deployment/payroll
kubectl rollout undo deployment/payroll
```

maxSurge and **maxUnavailable** control how aggressive that is. The defaults allow one extra pod and one missing pod, which is safe and slow.

7. Services

Pods come and go and their addresses change. A Service is the stable name in front of them.

```
apiVersion: v1
kind: Service
metadata:
  name: payroll
spec:
  selector:
    app: payroll
  ports:
    - port: 8080
      targetPort: 8080
```

Any pod in the cluster now reaches it at **payroll:8080**, or **payroll.default.svc.cluster.local** in full. That is the same idea as a Compose service name, at cluster scale.

```
kubectl get svc payroll
```

Output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
payroll	ClusterIP	10.96.184.22	<none>	8080/TCP	3m

That address belongs to the Service and does not change while it exists, which is the whole point of it.

Type	What it does
ClusterIP	Reachable inside the cluster only. The default
NodePort	Also opens a port on every node
LoadBalancer	Asks the cloud provider for a load balancer

port is the Service's port, **targetPort** is the container's. They are often the same, which hides the difference until the day they are not.

A Service finds pods by label, not by Deployment. Nothing connects the two objects except the labels matching, which is why a typo produces a Service with no endpoints and no error message.

```
kubectl get endpoints payroll    # empty means the selector matches nothing
```

8. ConfigMaps

A ConfigMap holds configuration that is not secret, so it can live in git beside the manifests.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: payroll-config
data:
  SPRING_PROFILES_ACTIVE: production
  LOG_LEVEL: INFO
```

There are two ways to use it, and the choice has consequences.

```
envFrom:
- configMapRef:
  name: payroll-config
```

```
volumeMounts:
  - name: config
    mountPath: /app/config
volumes:
  - name: config
    configMap:
      name: payroll-config
```

As environment variables, a change does nothing until the pods restart. The values were read at container start. As a mounted file the content is updated in place after a delay, but only an application that watches the file notices.

```
kubectl rollout restart deployment/payroll
```

That command exists because of this. It is the normal way to apply a changed ConfigMap.

9. Secrets

A Secret looks like a ConfigMap and is treated differently by the cluster.

```
apiVersion: v1
kind: Secret
metadata:
  name: payroll-secrets
type: Opaque
stringData:
  SPRING_DATASOURCE_PASSWORD: s3cr3t
```

```
envFrom:
  - secretRef:
      name: payroll-secrets
```

A Secret is base64, and base64 is not encryption. Anyone who can read the Secret can read the value, and

`kubectl get secret -o yaml` is one command. This is the question interviews actually ask.

```
kubectl get secret payroll-secrets -o jsonpath='{.data.SPRING_DATASOURCE_PASSWORD}' |
base64 -d
```

What makes Secrets safe is what is put around them:

Measure	What it gives
Encryption at rest	The value is encrypted in etcd rather than stored plainly
RBAC	Only some accounts can read Secrets at all
External store	Vault, AWS Secrets Manager or the CSI driver holds the real value

Never commit a Secret manifest to git. The usual answers are a sealed or encrypted manifest, or an operator that pulls the value from an external store at run time.

10. Resource requests and limits

```
resources:
  requests:
    cpu: "250m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
```

Requests are for scheduling. Limits are for enforcement. The request is what the scheduler reserves when it picks a node. The limit is what the kernel stops the container from exceeding.

The two limits behave differently, and this is the part worth remembering:

Over the limit	What happens
CPU	The container is throttled. It gets slower
Memory	The container is killed. <code>OOMKilled</code> , exit code 137

Set the memory request and limit to the same value for a JVM service. A pod that is scheduled as if it needs 512 MB and then grows to 1 GB gets killed on a busy node.

```
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-XX:MaxRAMPercentage=75"
```

A modern JVM reads the container limit rather than the node's memory, but takes a quarter of it by default. Without this, a 1 Gi limit gives the heap 256 MB and produces `OutOfMemoryError` inside a container that looks half empty.

11. Liveness and readiness probes

Two probes, two different questions. Kubernetes cannot tell a process that is still starting from one that is stuck, or a healthy process that is briefly busy from one that has failed, so it asks the application directly. Getting the two the wrong way round is one of the few configuration mistakes that can take a whole service down on its own.

Probe	Question	Failing means
Readiness	Can it serve traffic now?	Taken out of the Service
Liveness	Is it still working at all?	The container is restarted
Startup	Has it finished starting?	The other probes are held off

```
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
    periodSeconds: 5

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
    periodSeconds: 10

startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  failureThreshold: 30
  periodSeconds: 5
```

Liveness must not check dependencies. A liveness probe that queries the database restarts every pod when the database is slow, which removes the whole service at the worst possible moment. Readiness checks dependencies; liveness checks only that the process itself is still working.

A slow start needs a `startupProbe`, not a longer liveness period. A Spring Boot service taking forty seconds will otherwise fail liveness during startup and be restarted, forever. The `failureThreshold` above allows 150 seconds to start and then hands over to normal liveness checks.

12. Container restart behavior

A pod created by a Deployment has `restartPolicy: Always`. A container that exits is restarted in place, whatever the exit code.

Restarts back off. The delay doubles from 10 seconds to a maximum of five minutes, and resets after the container has run successfully for ten minutes.

```
kubectl get pods
```

Output:

NAME	READY	STATUS	RESTARTS	AGE
payroll-7d4b9c8f5d-x2plq	1/1	Running	0	4h
payroll-7d4b9c8f5d-m9kzt	0/1	CrashLoopBackOff	7 (2m ago)	22m

RESTARTS is the number worth watching. A pod that is **Running** with a restart count climbing is failing repeatedly and recovering, which no alert on pod status will catch.

Restarting the container is not rescheduling the pod. The pod stays on the same node with the same name; only the process inside is started again. Deleting the pod is what moves it.

13. ImagePullPolicy

Value	Behavior
Always	Contacts the registry every time a pod starts
IfNotPresent	Uses the node's copy if there is one
Never	Only uses a local image, never pulls

The default depends on the tag. A **:latest** tag defaults to **Always**; anything else defaults to **IfNotPresent**. That single rule explains a common complaint: a rebuilt image pushed under the same version tag does not appear, because the node already has one by that name and never looks again.

```
image: ghcr.io/acme/payroll:1.4
imagePullPolicy: Always
```

Deploying by digest makes the question disappear. A digest names exact bytes, so a cached copy is by definition the right one.

Never is for local clusters where the image was side-loaded with `kind load docker-image`.

14. ImagePullBackOff

The node tried to pull the image and could not. **ErrImagePull** is the first failure; **ImagePullBackOff** is the state it enters while retrying.

```
kubectl describe pod payroll-7d4b9c8f5d-m9kzt
```

The `Events` section at the bottom names the cause.

Cause	What it looks like
Wrong name or tag	<code>manifest unknown</code> , or <code>not found</code>
Private registry, no credentials	<code>unauthorized</code> , <code>authentication required</code>
Missing <code>imagePullSecrets</code>	The same, on an image that pulls fine locally
Docker Hub rate limit	<code>toomanyrequests</code>
Local-only image	<code>not found</code> for an image that exists on the laptop
Wrong architecture	<code>no match for platform</code>

The order to check them: read the events, then confirm the reference is exactly what `docker pull` would accept, then confirm the pull secret exists in the same namespace as the pod.

15. CrashLoopBackOff

The image pulled and the container started, and then it kept exiting. Kubernetes restarts it with a growing delay, which is the "backoff" in the name.

The container is not broken in Kubernetes. It is exiting, and the reason is in its own output.

```
kubectl logs payroll-7d4b9c8f5d-m9kzt --previous
```

`--previous` is the important flag. The current container may have only just started, so the logs that explain the failure belong to the one before it.

Cause	How it shows
Missing configuration	An exception at startup in the logs
Wrong database URL or password	A connection failure, then exit
Out of memory	<code>OOMKilled</code> in <code>describe</code> , exit code 137
Liveness probe too strict	Restarts on a schedule, logs look healthy
The command exits normally	Exit code 0, nothing wrong in the logs

```
kubectl describe pod payroll-7d4b9c8f5d-m9kzt | grep -A5 'Last State'
```

Exit code 0 in a `CrashLoopBackOff` means the container finished. It is not a crash at all. Something is running a one-off command where a long-lived process is expected, and the restart policy is doing exactly what it was told. Work that is meant to finish belongs in a Job, not a Deployment.

To look around inside a container that will not stay up, replace its command so it does nothing and stays running, then `exec` into it.

```
kubectl run debug --image=ghcr.io/acme/payroll:1.4 \
  --command -- sleep 3600
kubectl exec -it debug -- sh
```

The pieces together

Object	What it answers
Pod	What actually runs
Deployment	How many, and which image
ReplicaSet	The generation the Deployment created
Service	The stable name in front of the pods
ConfigMap	Configuration that is not secret
Secret	Configuration that is, with protection around it
Requests	What the scheduler reserves
Limits	What the kernel enforces
Readiness	Whether traffic is sent
Liveness	Whether the container is restarted
<code>imagePullPolicy</code>	Whether the node looks at the registry again

For example, a release. The pipeline pushes `payroll@sha256:4a1c8f...` and `kubectl set image` puts that digest on the Deployment. A new ReplicaSet is created, the first pod starts, and the Service does not send it traffic until the readiness probe passes. If the pod cannot pull the image the status reads `ImagePullBackOff` and `describe` names the reason; if it pulls and exits the status reads `CrashLoopBackOff` and `logs --previous` holds the exception. If neither happens, the old pods are removed and the rollout finishes.

Two commands answer most questions here, and they answer them in order: `kubectl describe pod` for anything that happened before the container started, and `kubectl logs --previous` for anything after. Knowing which of the two to reach for is most of the skill.